



Module 5

Convolutional Neural Networks



DL: Convolutional Nets

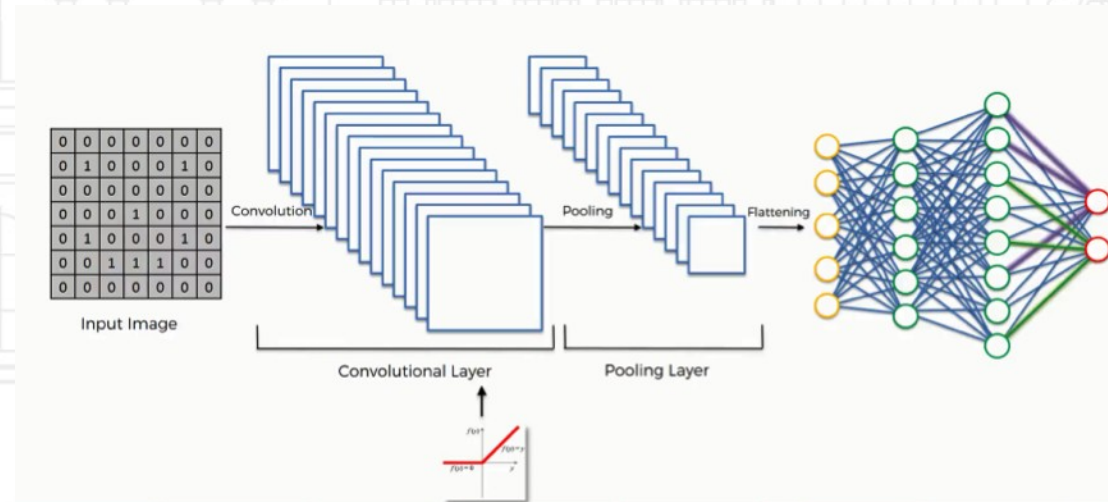


Image Representation

- ❑ We know images are represented by tensors
- ❑ A colored pixel is represented as three separate values (RGB).
- ❑ 3 matrices (tensor) represent red (R), green (G), blue (B) channels.

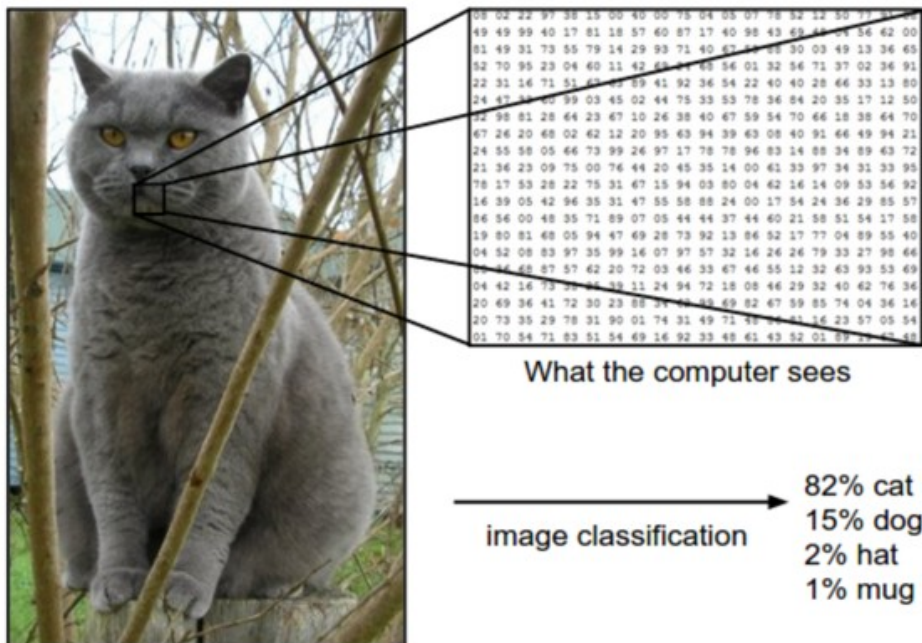


Image Representation

- ❑ Channels can be flattened to create a single vector.

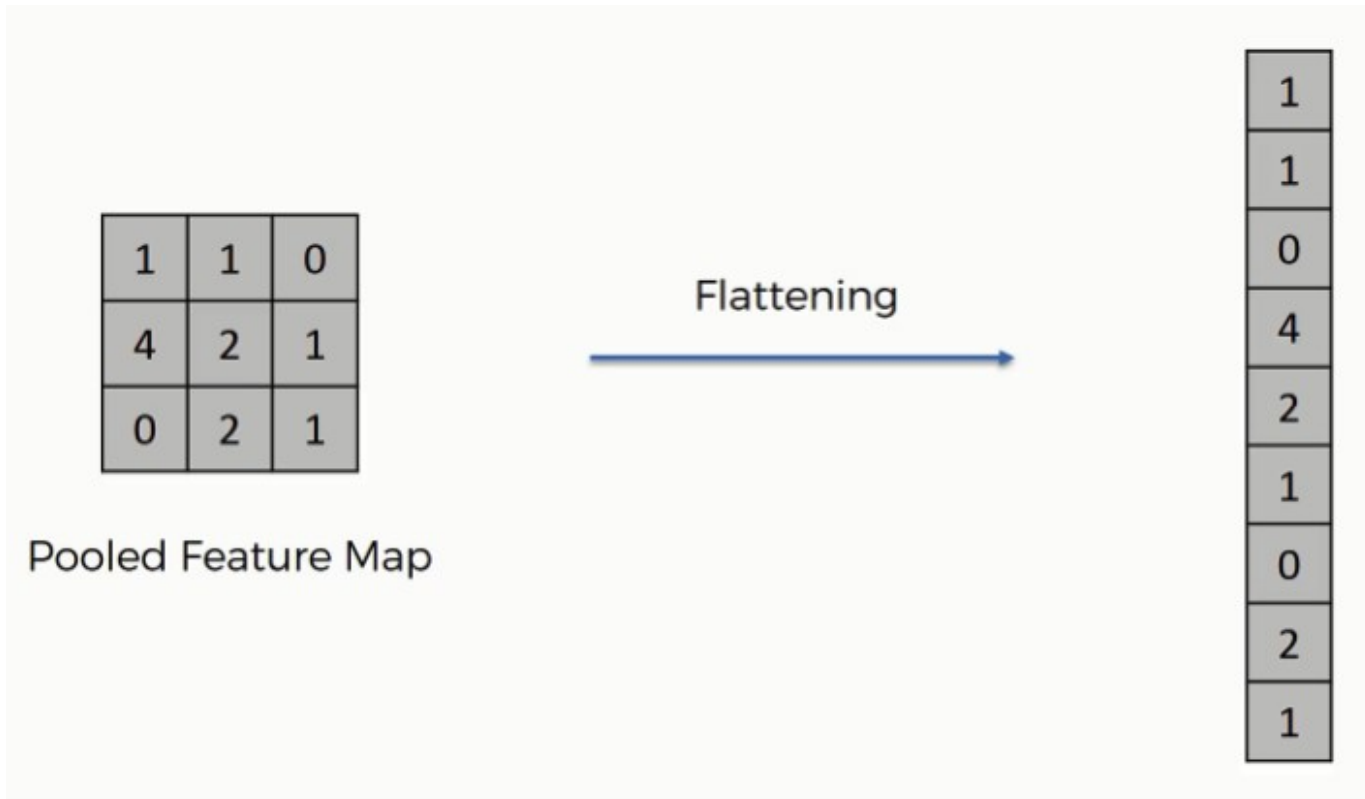
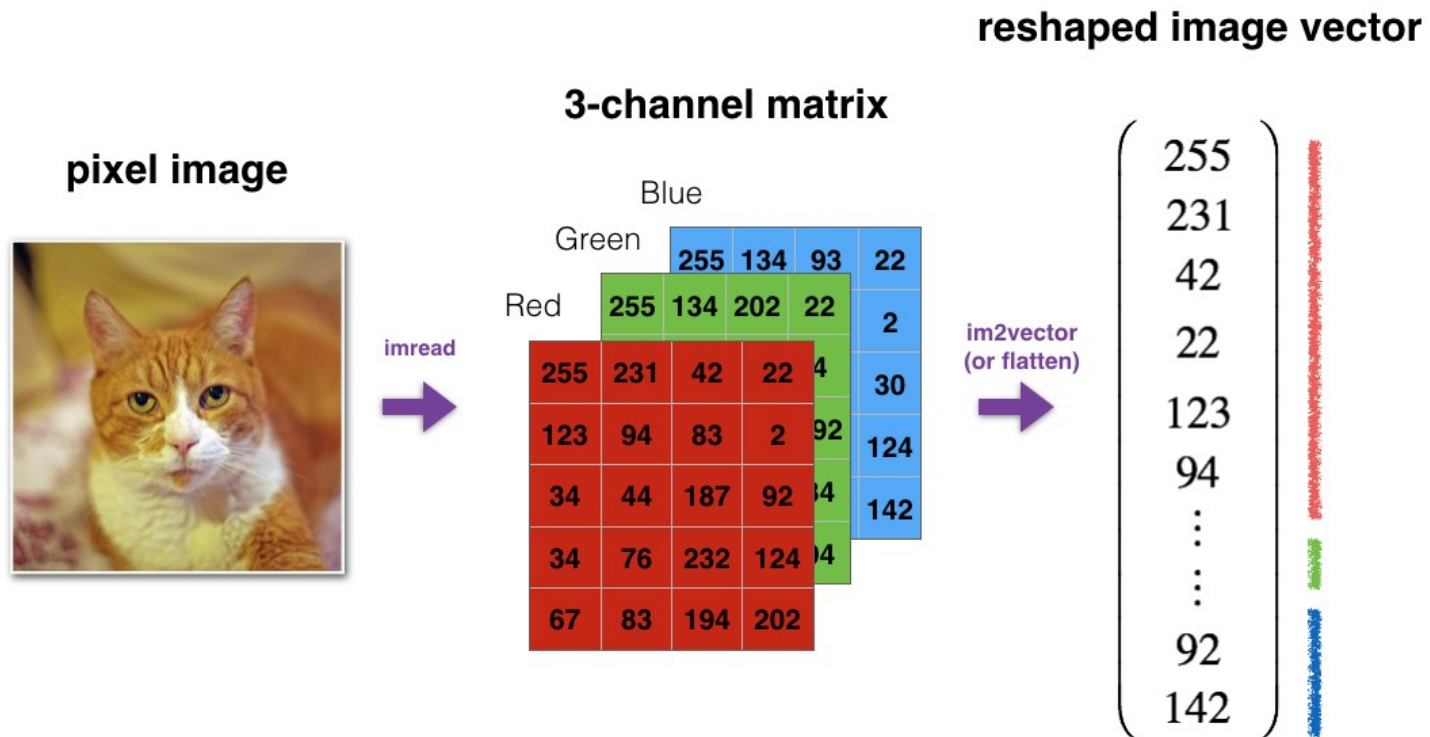


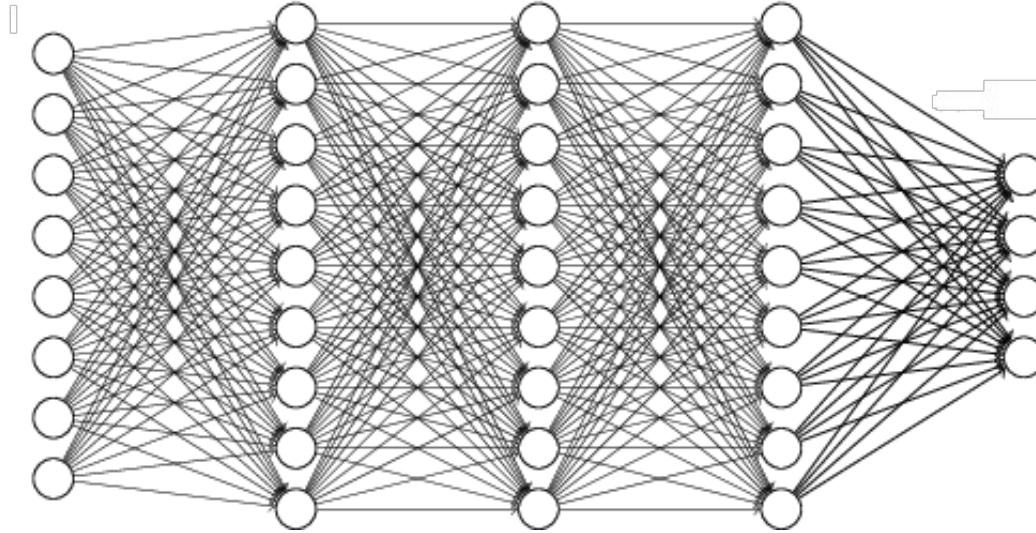
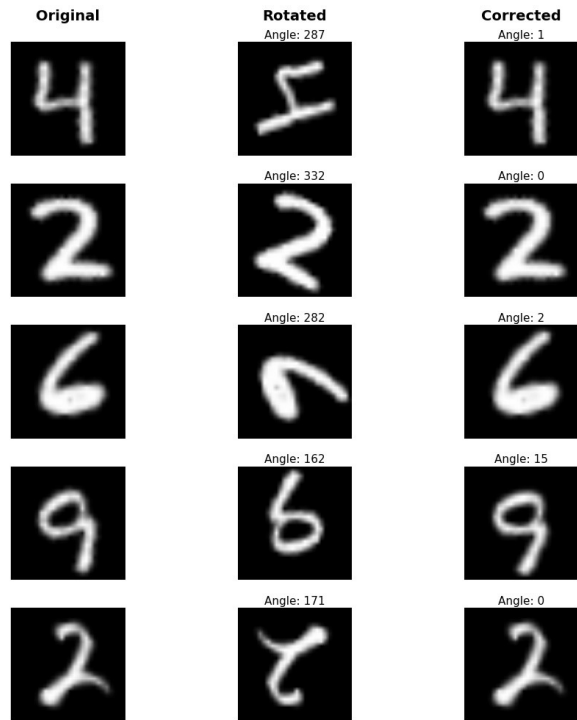
Image Representation

- Channels can be flattened to create a single vector.



Using Feed-Forward NNs for Images

- ❑ Feed forward NNs are not very effective with images



Using Feed-Forward NNs for Images

❑ Translation Invariance



Cat



Cat

Using Feed-Forward NNs for Images

- ❑ Other Invariances cause issues

Translation Invariance



Rotation/Viewpoint Invariance



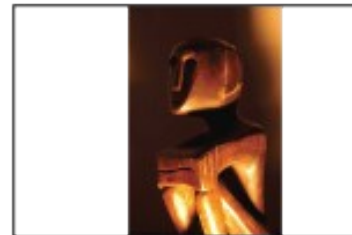
Using Feed-Forward NNs for Images

- ❑ Other Invariances cause issues

Size Invariance

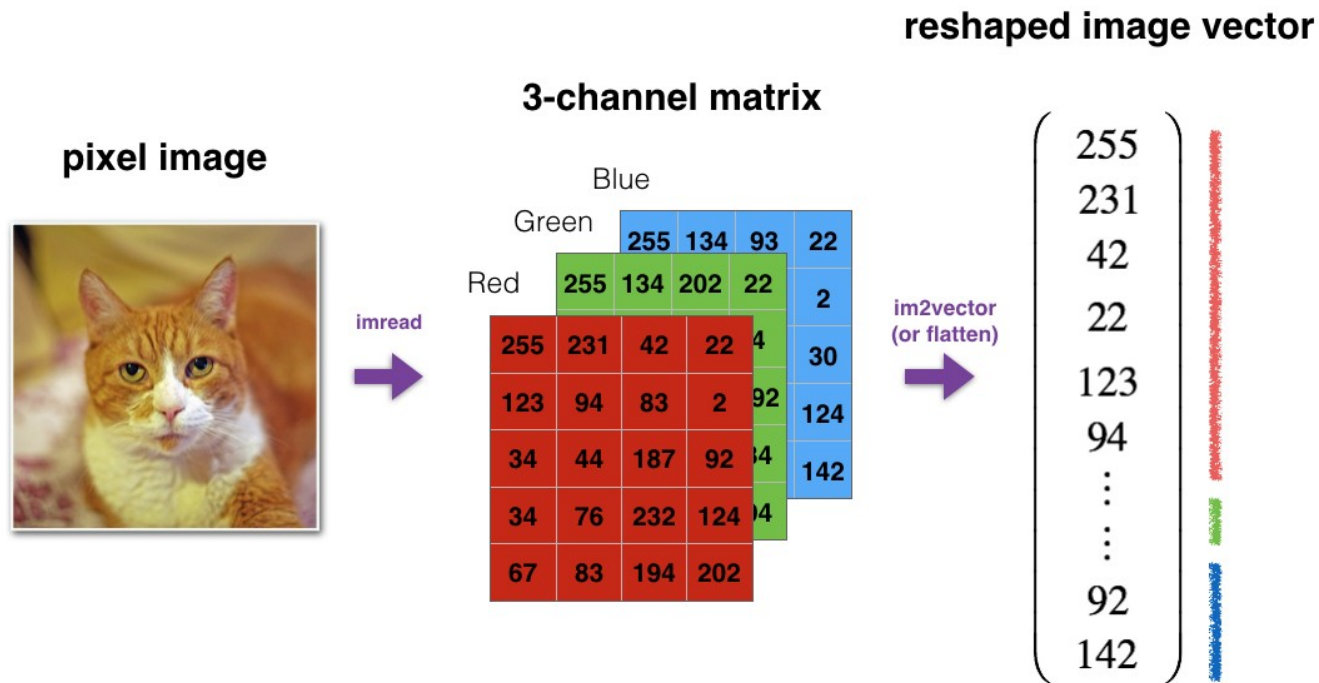


Illumination Invariance



Using Feed-Forward NNs for Images

- ❑ Flattening tensors create an enormous amount of features.
- ❑ Predictive power declines with the added parameters estimated.





Python

Convolutions (think about Waldo)



Convolutions

- A convolution (kernel, mask, filter) is a small matrix used for blurring, sharpening, embossing, edge detection

Input Kernel Output

0	1	2
3	4	5
6	7	8

*

0	1
2	3

=

19	25
37	43

$$\begin{aligned} 0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 &= 19, \\ 1 \times 0 + 2 \times 1 + 4 \times 2 + 5 \times 3 &= 25, \\ 3 \times 0 + 4 \times 1 + 6 \times 2 + 7 \times 3 &= 37, \\ 4 \times 0 + 5 \times 1 + 7 \times 2 + 8 \times 3 &= 43. \end{aligned}$$

Convolutions

- ❑ Convolutions go over the image and produce an output

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

=

6		

$7 \times 1 + 4 \times 1 + 3 \times 1 +$
 $2 \times 0 + 5 \times 0 + 3 \times 0 +$
 $3 \times -1 + 3 \times -1 + 2 \times -1$
 $= 6$

Convolutions

- Suppose we have an image $I_{n \times p}$ and a kernel $K_{k \times l}$
- The resulting image $O_{n-k+1} \times p-l+1$

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

=

6		

$$\begin{aligned}
 &7 \times 1 + 4 \times 1 + 3 \times 1 + \\
 &2 \times 0 + 5 \times 0 + 3 \times 0 + \\
 &3 \times -1 + 3 \times -1 + 2 \times -1 \\
 &= 6
 \end{aligned}$$

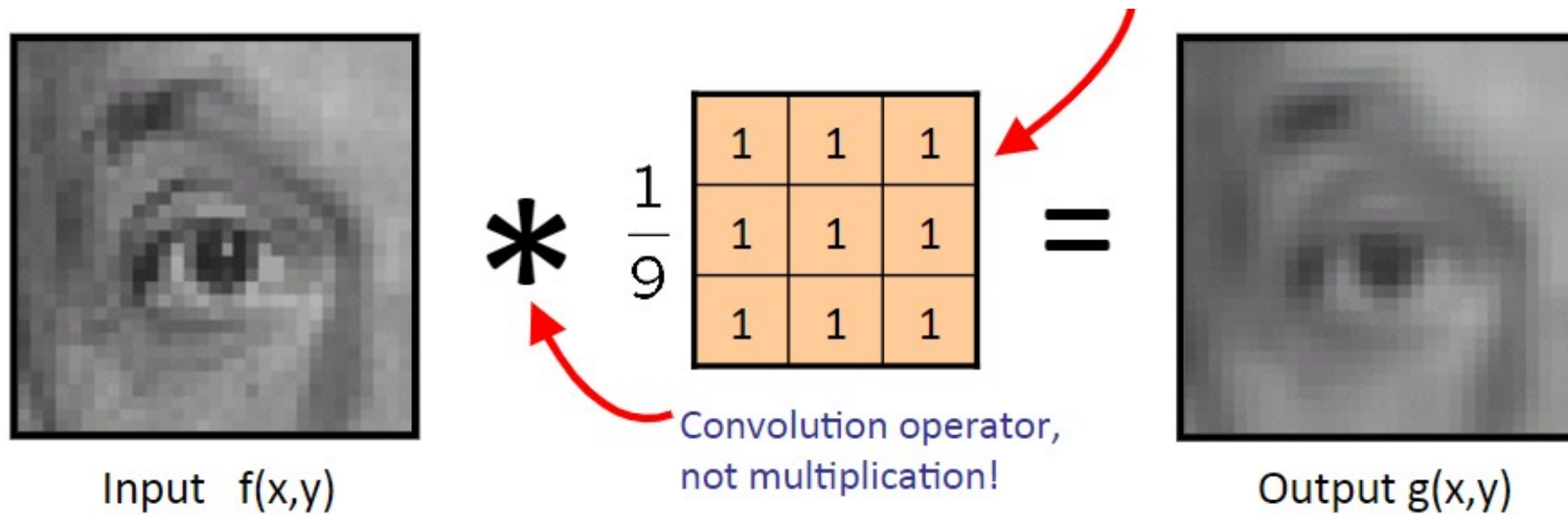
$$I_{5 \times 5}$$

$$K_{3 \times 3}$$

$$O_{5-3+1 \times 5-3+1} = O_{3 \times 3}$$

Convolutions Use

- ❑ Convolutions can blur (downsample) images



Convolutions Use

- ❑ We can create multiple levels of blurring
- ❑ In other words, create multiple images of the same one.

Input Image

*

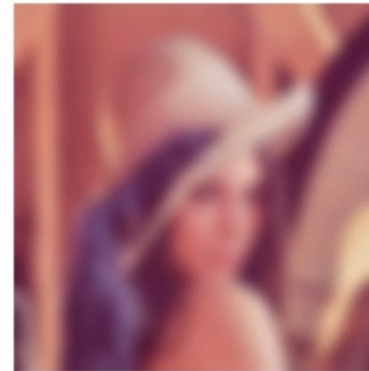
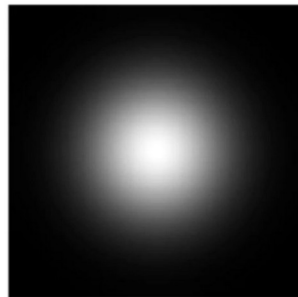
Filter (Kernel)

=

Output Image



*

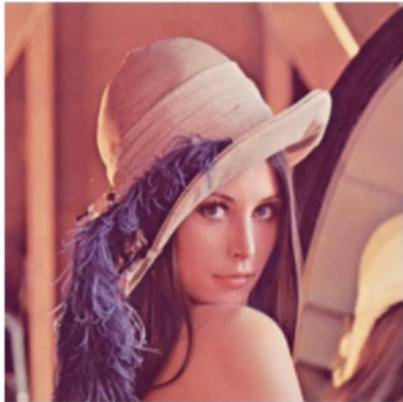


$$\begin{pmatrix} 0.0625 & 0.125 & 0.0625 \\ 0.125 & 0.25 & 0.125 \\ 0.0625 & 0.125 & 0.0625 \end{pmatrix}$$

Convolutions Use

- We can also better detect edges.

Input Image



*

Filter (Kernel)

=

Output Image

$$\sqrt{G_x^2 + G_y^2}$$

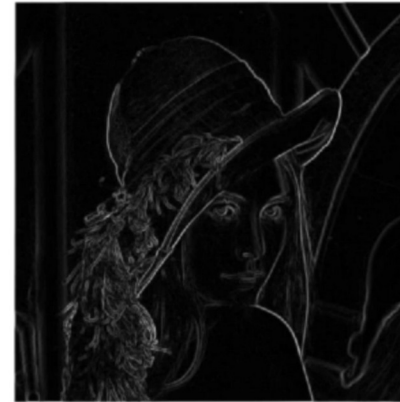
*

$G_x =$

-1	0	1
-2	0	2
-1	0	1

$G_y =$

1	2	1
0	0	0
-1	-2	-1



Convolutions Use

- ❑ Convolutions work to create new features.



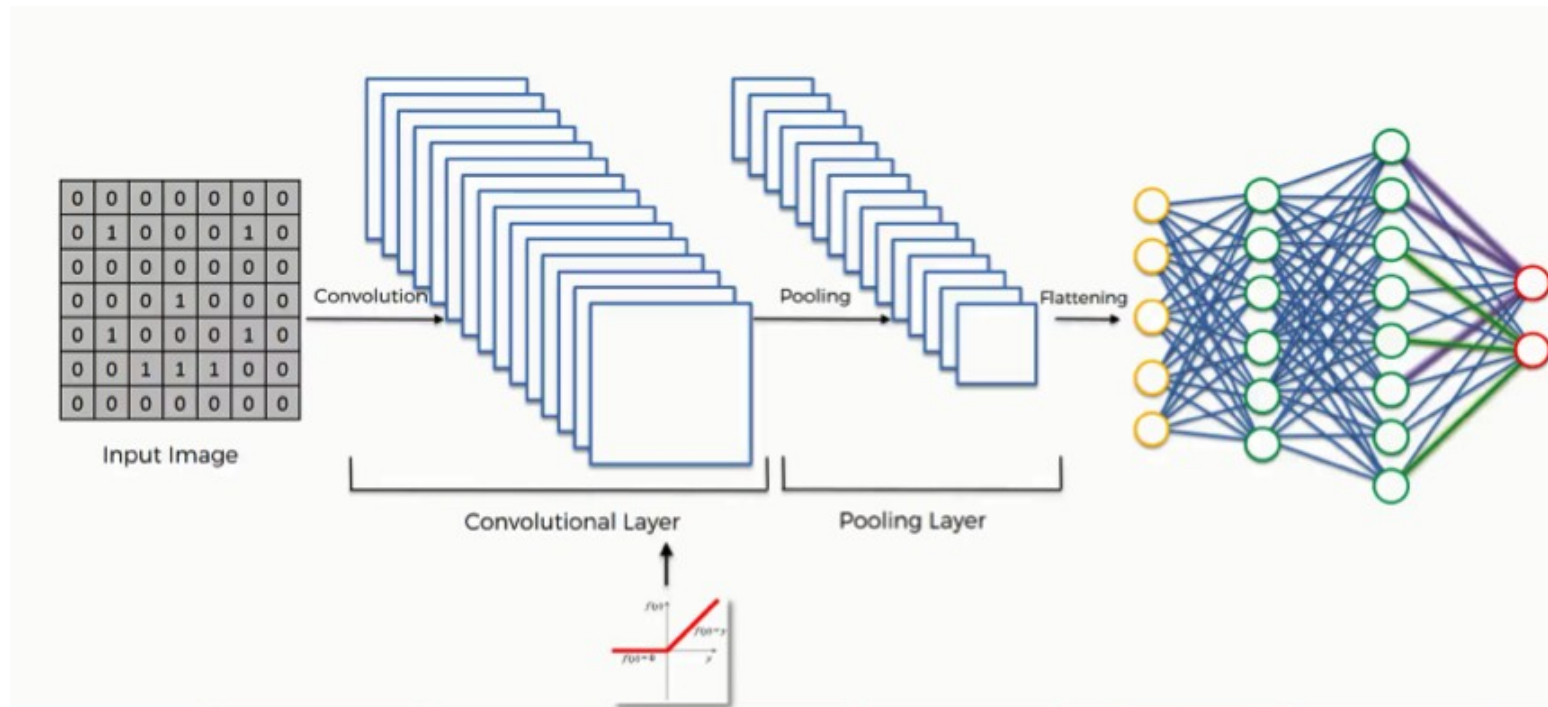
Convolutions Use

- When we combine multiple convolutions we create a convolutional layer.



Convolutions Use

- When we combine multiple convolutions we create a convolutional layer.



Example

- Suppose we have the following image and convolution.

Image

0	10	10	0
20	30	30	20
10	20	20	10
0	5	5	0

*

Filter

1	0
0	2

slido



What is the resulting output image shape

□ Start presenting to display the poll results on this slide.

Example

- Suppose we have the following image and convolution.

Image * Filter = Output

0	10	10	0
20	30	30	20
10	20	20	10
0	5	5	0

Filter

1	0
0	2

Output

slido



What is the value of X?

□ Start presenting to display the poll results on this slide.

Example

- Suppose we have the following image and convolution.

Image	*	Filter	=	Output																													
<table><tr><td>0</td><td>10</td><td>10</td><td>0</td></tr><tr><td>20</td><td>30</td><td>30</td><td>20</td></tr><tr><td>10</td><td>20</td><td>20</td><td>10</td></tr><tr><td>0</td><td>5</td><td>5</td><td>0</td></tr></table>	0	10	10	0	20	30	30	20	10	20	20	10	0	5	5	0		<table><tr><td>1</td><td>0</td></tr><tr><td>0</td><td>2</td></tr></table>	1	0	0	2		<table><tr><td>60</td><td>70</td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>	60	70							
0	10	10	0																														
20	30	30	20																														
10	20	20	10																														
0	5	5	0																														
1	0																																
0	2																																
60	70																																

$$1*10 + 0*10 + 0*30 + 2*30 = 70$$

Example

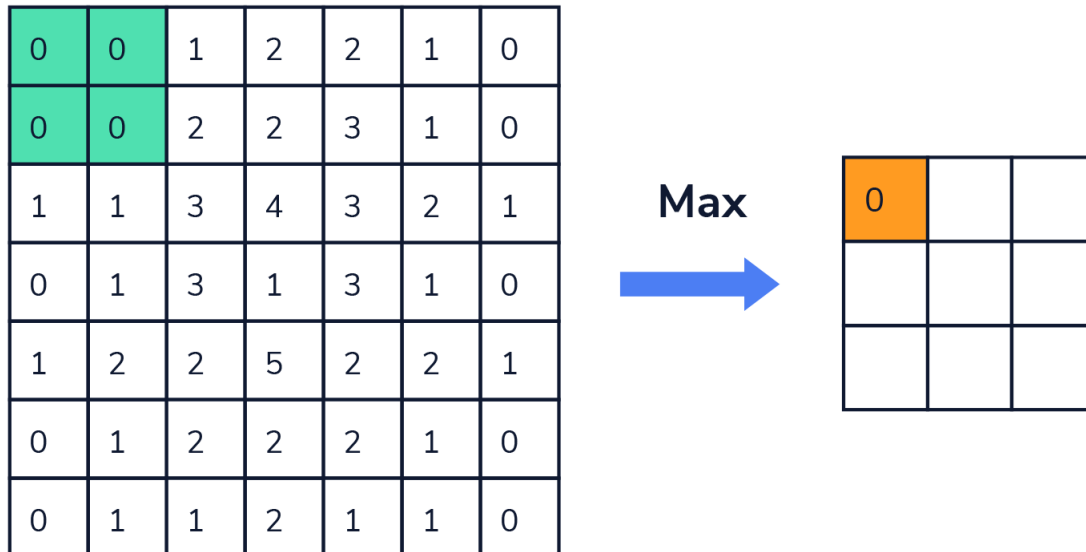
- Suppose we have the following image and convolution.

Image	*	Filter	=	Output																													
<table><tr><td>0</td><td>10</td><td>10</td><td>0</td></tr><tr><td>20</td><td>30</td><td>30</td><td>20</td></tr><tr><td>10</td><td>20</td><td>20</td><td>10</td></tr><tr><td>0</td><td>5</td><td>5</td><td>0</td></tr></table>	0	10	10	0	20	30	30	20	10	20	20	10	0	5	5	0		<table><tr><td>1</td><td>0</td></tr><tr><td>0</td><td>2</td></tr></table>	1	0	0	2		<table><tr><td>60</td><td>70</td><td>50</td></tr><tr><td>60</td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>	60	70	50	60					
0	10	10	0																														
20	30	30	20																														
10	20	20	10																														
0	5	5	0																														
1	0																																
0	2																																
60	70	50																															
60																																	

$$1*20 + 0*30 + 0*10 + 2*20 = 60$$

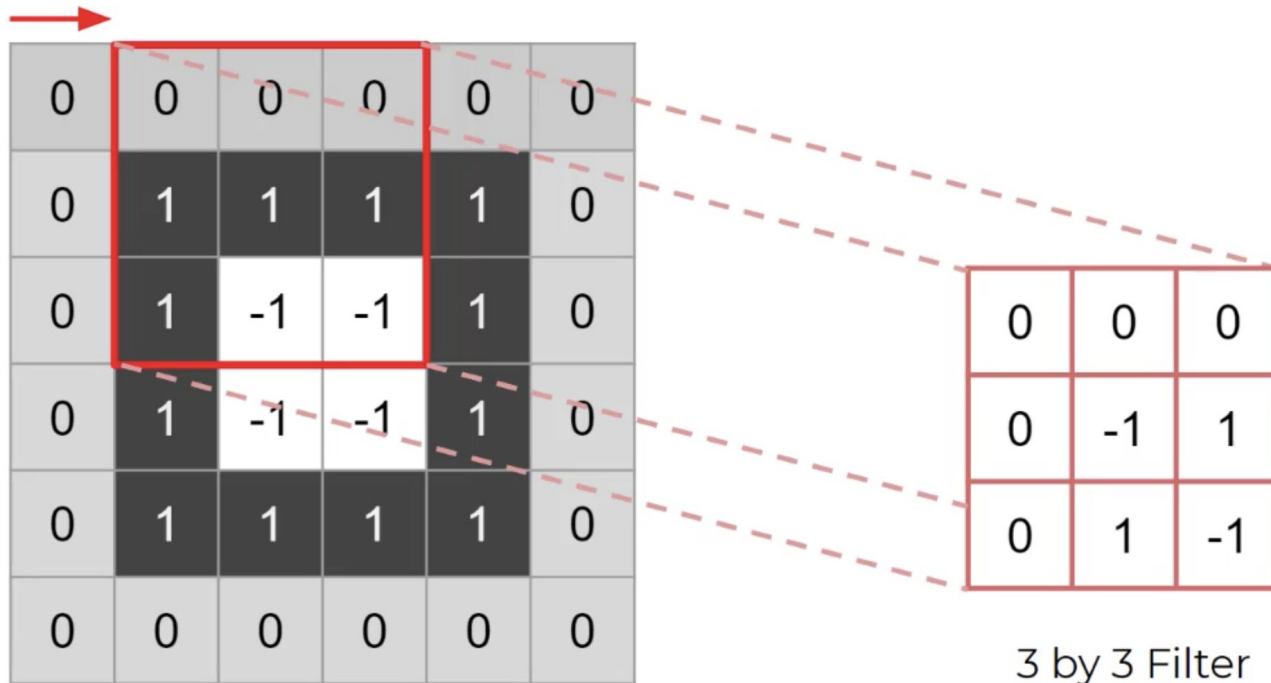
Pooling Filter

- ❑ Pooling is also used to downsample the images
- ❑ Pooling filters keep the important parts of the images.
- ❑ Max, Min and Average Pooling Filters are the most common



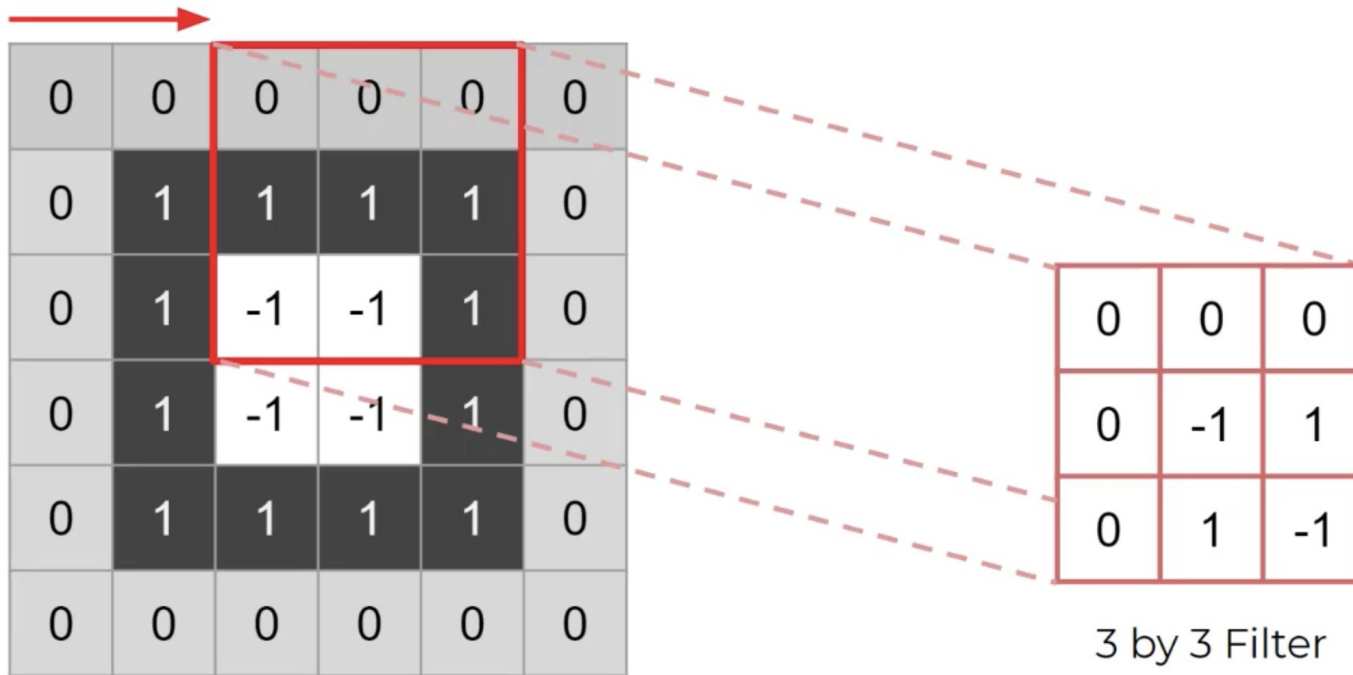
Stride

- ❑ Dictates the movement of the kernel, or filter, across the image
- ❑ Example of (1,1) stride (most common)



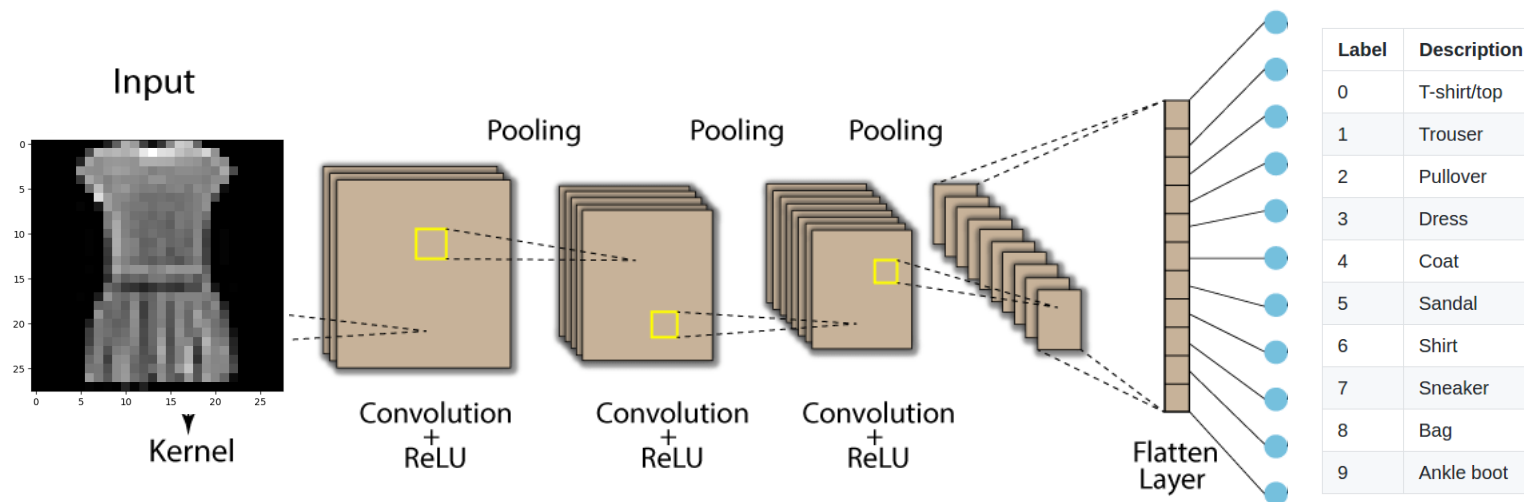
Stride

- Example of a (2,2) stride



Final CNN Product

- ❑ We combine multiple convolutional layers and pooling layers
- ❑ We just set the size of the kernels
- ❑ We let the network optimize values of the filters
- ❑ We flatten and add a feed-forward neural net for further accuracy.



Final CNN Product

- ❑ We combine multiple convolutional layers and pooling layers
- ❑ We just set the size of the kernels
- ❑ We let the network optimize values of the filters
- ❑ We flatten and add a feed-forward neural net for further accuracy.

